PDPM: Perceptron-Driven Pipeline Morphing for Adaptive RISC-V Processors

Dawit Chun[1] and Han-seok Ko[2]
[1]Taejae University, Seoul, South Korea
[2]The Catholic University of America, Washington, DC, USA
dawitchun@taejae.ac.kr, koha@cua.edu

***Abstract*—We present PDPM (Perceptron-Driven Pipeline Morphing), a morphable RISC-V processor that dynamically reconfigures its pipeline between 3-stage (P3), 5-stage (P5), and 7-stage (P7) depths, each with mode-appropriate clock scaling, creating a 12-configuration optimization space. An online perceptron predictor jointly selects the optimal pipeline depth and clock frequency from three runtime workload features—branch rate, CPI ratio, and stall percentage—learning during execution with no prior training data. Extending Jiménez and Lin's perceptron branch predictor to structural microarchitectural decisions, PDPM achieves 0.560–3.698% overhead versus a perfect oracle across 108 benchmarks (100 assembly-compiled, 8 C-compiled), compared to 34% for a fixed P5@1.0× baseline—a 14× reduction in wasted performance for under 1% hardware cost (fewer than 50 LUT4 cells). On 78 unseen benchmarks after training on just 30, the predictor achieves 97.436% configuration accuracy with +0.003% overhead, capturing 99.997% of oracle-optimal performance. The learned 12×4 weight matrix is fully interpretable: the perceptron independently discovers that high branch rate favors P3@1.1×, high stall rate favors P5@1.3×, and low CPI ratio favors P7@1.5×—recovering known architectural principles without explicit programming. The complete system is implemented in 4,000 lines of Verilog and validated on a Lattice iCE40 UP5K FPGA at 85% logic utilization (4,506 logic cells) with a maximum clock frequency of 52.96 MHz.**

***Index Terms*—Morphable pipeline, RISC-V, adaptive microarchitecture, online learning, perceptron predictor, pipeline depth optimization, DVFS, FPGA**

I. Introduction

THE depth of a processor pipeline determines a fundamental tradeoff: deeper pipelines enable higher clock frequencies by shortening the critical path per stage, but they incur severe penalties when branches or data hazards disrupt sequential flow. This tension, explored extensively since Intel's 31-stage Pentium 4 NetBurst architecture [1], has no universally optimal resolution. The best pipeline depth depends on workload characteristics that vary not only between programs but within a single program's execution phases. A branch-heavy parser benefits from a shallow, fast-flushing pipeline, while a compute-heavy matrix kernel benefits from a deep pipeline running at maximum clock frequency.

Prior work has addressed this through static analysis [2], [9], dynamic voltage-frequency scaling (DVFS) [3], [4], and adaptive pipeline structures [5], [6]. However, these approaches either fix the pipeline depth at design time, adapt only clock frequency without changing pipeline structure, or require complex hardware for width reconfiguration. No existing system jointly optimizes both pipeline depth *and* clock frequency using online machine learning that generalizes to unseen workloads.

We introduce PDPM (Perceptron-Driven Pipeline Morphing), a morphable RISC-V processor that addresses this gap. The system dynamically reconfigures between three pipeline depths—P3 (3-stage, 1-cycle branch penalty), P5 (5-stage, 2-cycle branch penalty), and P7 (7-stage, 5-cycle branch penalty)—each with mode-appropriate clock scaling ranging from 1.1× (P3) to 1.5× (P7) base frequency, yielding 12 valid configurations. An online perceptron predictor, inspired by Jiménez and Lin's neural branch prediction [7], learns the mapping from runtime workload features to the optimal configuration during execution.

The contributions of this work are fourfold. First, we present a morphable 3/5/7-stage RISC-V pipeline using bypass multiplexers within a superset architecture, implemented in 4,000 lines of Verilog and validated on a Lattice iCE40 UP5K FPGA at 85% logic utilization (4,506 logic cells) with a maximum clock frequency of 52.96 MHz. Second, we introduce a joint pipeline-depth and clock-frequency online perceptron predictor that learns optimal configurations from workload features during execution, extending perceptron-based prediction from branch direction to structural microarchitectural decisions—a 12-class problem solved with only 48 bytes of learned weights. Third, we evaluate across 108 benchmarks—100 assembly-compiled and 8 C-compiled using `riscv-none-elf-gcc`—demonstrating that PDPM reduces performance waste from 34% (fixed baseline) to 0.560–3.698% (online perceptron), with 97.436% generalization accuracy on 78 unseen benchmarks. Fourth, we show that the learned 12×4 weight matrix is interpretable and independently recovers known architectural decision boundaries without explicit programming.

II. Related Work

A. Pipeline Depth Optimization

The relationship between pipeline depth and performance has been studied since the earliest pipelined machines. Kunkel and Smith [14] provided an early analytical framework showing that optimal depth depends on technology-specific latch overhead and logic delay. Hrishikesh et al. [2] established that the optimal logic depth per pipeline stage is 6–8 FO4 inverter delays, a result that influenced the design of deep-pipeline processors including the Pentium 4. Sprangle and Yale [15] analyzed the diminishing returns of increasing pipeline depth, demonstrating that branch misprediction penalties grow linearly with depth while clock frequency gains are sublinear.

Intel's Pentium 4 NetBurst architecture [16] pushed pipeline depth to 20+ stages (31 in Prescott), achieving high clock

frequencies but suffering significant performance penalties on branch-heavy workloads. The subsequent retreat to shorter pipelines in the Core architecture [1] demonstrated that deeper is not always better—a lesson that motivates runtime adaptation. Hartstein and Puzak [9] developed a comprehensive analytical model relating pipeline depth to IPC and clock frequency, identifying an optimal depth that depends on cache miss rates and branch misprediction penalties. Their model, however, assumes a fixed depth chosen at design time; PDPM makes this depth a runtime-configurable parameter.

B. Adaptive and Reconfigurable Microarchitectures

Albonesi [5] proposed selective cache ways for dynamic resource allocation, establishing the principle that microarchitectural structures can be resized at runtime for power efficiency. Bahar and Manne [6] investigated dynamic pipeline width control, showing that reducing issue width during low-ILP phases saves power with minimal performance loss. Lozano and Gao [17] proposed elastic pipelines using FIFO-based interstage communication that allow stages to operate at different rates, and Moshnyaga and Ikenaga [19] studied pipeline stage unification for low-power embedded processors.

More recently, Lukefahr et al. [20] proposed composite cores that dynamically switch between a big core and a little core within the same processor, sharing architectural state. While conceptually related to PDPM, composite cores switch between entire microarchitectures rather than reconfiguring a single pipeline's depth, requiring substantially more silicon area. ARM's big.LITTLE architecture [21] implements a similar concept at the system level with heterogeneous cores, and Jacobson et al. [18] investigated dynamically configurable processors that adapt multiple structures simultaneously. PDPM is distinguished from all these approaches by its use of online feature-based learning rather than heuristic-based or threshold-based switching.

C. Dynamic Voltage and Frequency Scaling

DVFS is the most widely deployed form of runtime adaptation. Kim et al. [3] demonstrated per-core DVFS guided by workload characteristics, while Eyerman and Eeckhout [4] developed a governor based on leading loads that identifies memory-bound phases benefiting from frequency reduction. Isci et al. [30] proposed phase-aware DVFS using workload classification. PDPM differs from DVFS in a fundamental way: we jointly optimize pipeline *structure* and frequency rather than frequency alone. The optimal clock frequency depends on the chosen pipeline depth—a shallow pipeline's critical path is longer, limiting its maximum frequency—creating an interdependent two-dimensional optimization that pure DVFS cannot address.

D. Machine Learning in Microarchitecture

The application of machine learning to microarchitectural decisions was pioneered by Jiménez and Lin [7], whose perceptron branch predictor demonstrated that simple linear models can outperform complex table-based predictors for hard-to-predict branches. This work is the most direct ancestor of PDPM: we extend the same perceptron architecture from predicting branch direction (a binary decision) to predicting microarchitectural configuration (a 12-class decision over pipeline depth $\times$ clock frequency). Subsequent work extended perceptrons to path-based prediction [8] and combined them with traditional predictors in hybrid schemes [22]. Beyond branch prediction, machine learning has been applied to prefetching [23], cache replacement [24], resource allocation [25], and coordinated multicore management [27]. Nemirovsky et al. [26] surveyed machine learning techniques for computer architecture, identifying online learning as a particularly promising direction for runtime adaptation due to its ability to handle non-stationary workloads without retraining.

E. Multi-Armed Bandits for Online Configuration

The multi-armed bandit framework [28] provides an alternative to supervised learning for online configuration selection, and Dubach et al. [29] applied machine learning with performance counters to guide compiler optimization in a similar spirit. However, bandits require trying each arm independently, which scales poorly with large configuration spaces. Our evaluation (Section VI) compares PDPM against UCB1 [28] with 12 arms: on 78 unseen workloads, UCB1 achieves +45% overhead while PDPM achieves +0.003% overhead with 97.436% accuracy, demonstrating the advantage of feature-based generalization over trial-based exploration.

III. Performance Model and System Architecture

A. Analytical Performance Model

For a program of N dynamic instructions, execution time follows the Iron Law [10]:

$$T = \frac{N \times \text{CPI}}{f} \tag{1}$$

where CPI is cycles per instruction and f is clock frequency. Both depend on pipeline depth d, making the optimization nontrivial. We decompose CPI as:

$$\text{CPI}(d) = 1.0 + B \cdot P(d) + H \cdot S(d) + C_{\text{mem}} \tag{2}$$

where B is the fraction of taken branches, $P(d)$ is the branch penalty in cycles, H is the hazard frequency, $S(d)$ is stall cycles per hazard, and C_{mem} captures memory latency. From our implementation, the measured branch penalties are $P(3) = 1$, $P(5) = 2$, $P(7) = 5$ cycles.

Clock frequency scales with depth via $f(d) = f_{\text{base}} \cdot \alpha(d)$, where $\alpha(3) = 1.1$, $\alpha(5) = 1.3$, $\alpha(7) = 1.5$ in our FPGA implementation, reflecting measured critical path differences. Defining effective time as $E(d) = \text{CPI}(d)/\alpha(d)$ and assuming hazard and memory terms are small relative to branch effects:

$$E(\text{P3}) = (1 + B)/1.1 \tag{3}$$

$$E(\text{P5}) = (1 + 2B)/1.3 \tag{4}$$

$$E(\text{P7}) = (1 + 5B)/1.5 \tag{5}$$

Setting $E(\text{P3}) = E(\text{P5})$ yields the P3–P5 crossover at $B^*_{35} = 22\%$, and $E(\text{P5}) = E(\text{P7})$ yields the P5–P7 crossover

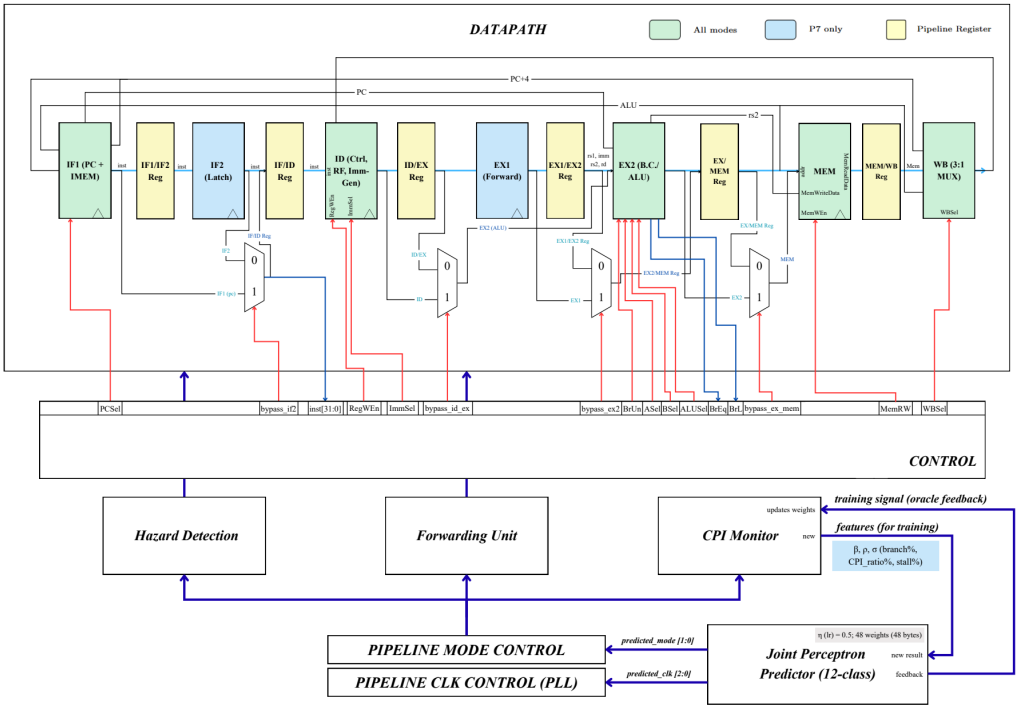

Fig. 1: System architecture of PDPM. The 7-stage superset datapath contains all pipeline stages; four bypass multiplexers (dashed paths) short-circuit stages in P3 and P5 modes. The online perceptron predictor observes hardware performance counters (branch rate, CPI ratio, stall rate) and jointly selects pipeline depth and clock frequency through the mode controller. The CPI monitor provides the training signal by comparing measured effective throughput across modes.

at $B^*_{57} = 8\%$. These analytical boundaries predict that P7 is optimal below 8% branch rate, P5 between 8–22%, and P3 above 22%. As we demonstrate in Section VI, the perceptron independently learns boundaries at approximately $\beta < 10\%$ for P7 and $\beta > 25\%$ for P3—closely matching these analytical predictions without being given the equations.

The decision boundaries are also highly sensitive to achievable clock ratios. If P3 runs at scaling factor α_3 and P5 at α_5, P3 is preferred when:

$$B > \frac{\alpha_5 - \alpha_3}{2\alpha_3 - \alpha_5} \tag{6}$$

For ideal scaling ($\alpha_3 = 1.0$, $\alpha_5 = 2.0$), this requires $B > 100\%$—impossible, meaning P3 would never be competitive. But with our measured FPGA constraints ($\alpha_3 = 1.1$, $\alpha_5 = 1.3$), P3 wins when $B > 22\%$, which many real workloads exceed. A mere 10% change in α_5 (from 1.3 to 1.43) would shift this boundary to $B > 43\%$, rendering P3 almost never optimal. This extreme sensitivity is precisely why a *learned* predictor that adapts to measured performance outperforms analytically derived static thresholds.

B. Superset Pipeline Design

Rather than implementing three independent pipelines, we implement the deepest configuration (P7) as a superset and use four bypass multiplexers to short-circuit stages for shallower modes (Fig. 1). When a bypass signal is asserted, the corresponding multiplexer routes data around the pipeline register, effectively removing that stage. In P3 mode all four bypasses are active: IF2 and EX2 are skipped, ID and EX are merged, and MEM and WB are merged, yielding a 3-stage pipeline with a single-cycle branch penalty. In P5 mode, only IF2 and EX2 are bypassed, yielding the classical 5-stage RISC-V pipeline with a 2-cycle branch penalty. P7 mode activates all seven stages with a 5-cycle branch penalty but enables the highest clock frequency (1.5× base). Table I summarizes the bypass configuration.

TABLE I: Bypass Signal Configuration for Each Pipeline Mode

Mode	bp_if2	bp_id_ex	bp_ex2	bp_ex_mem	Stages
P3	1	1	1	1	3
P5	1	0	1	0	5
P7	0	0	0	0	7

The seven stages are shown in Fig. 2. IF1 handles PC management and instruction memory access. IF2 (P7 only) registers the instruction output to break the fetch critical path. ID performs opcode decoding, register file read, and immediate generation. EX1 executes ALU operations and resolves operand forwarding. EX2 (P7 only) provides an additional execute cycle where branches resolve at deeper pipeline position. MEM interfaces with data memory, and WB writes results to the register file. Stages IF2 and EX2 contain only pipeline registers, enabling clean bypass without affecting datapath functionality.

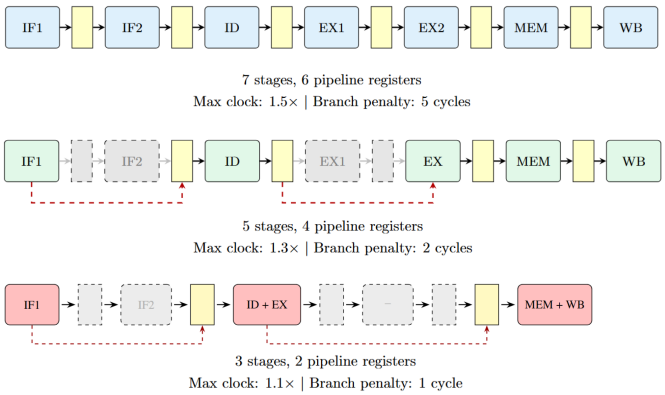

Fig. 2: Pipeline stage comparison across P3, P5, and P7 modes. P3 merges ID+EX and MEM+WB for minimum branch penalty (1 cycle). P7 activates all stages for maximum clock (1.5×) but incurs a 5-cycle branch penalty.

C. Forwarding, Hazard Detection, and Mode Switching

The forwarding unit uses a priority-encoded scheme with four sources: no forwarding (00), MEM/WB stage (01), EX/MEM stage (10), and an additional EX2 path (11) active only in P7 mode. The hazard detection unit monitors load-use dependencies and inserts stall cycles when the instruction in the execute stage reads from memory and its destination matches a source register in the decode stage.

A key engineering challenge was ensuring correct forwarding for JALR (function return) instructions across all three modes. In P5 and P7, the JALR instruction has moved past the decode stage when it resolves, so the return address source must be derived from the execute-stage `a_sel` signal rather than the decode-stage `is_jalr` flag. JAL sets `a_sel=1` (PC-relative target), while JALR sets `a_sel=0` (register-relative target). This distinction must be tracked through an additional pipeline register in P7 mode (Listing 1). This subtle bug caused all 8 C-compiled benchmarks to produce incorrect results in P5 and P7 modes until corrected, because every function return (`ret = jalr x0, ra, 0`) jumped to a wrong address computed from the branch target instead of the return address register.

Listing 1: JALR detection fix: deriving `is_jalr` from the correct pipeline stage in P5/P7 modes

```
// Register a_sel for P7 EX2 stage
reg ex2_a_sel_reg;
always @(posedge clk or posedge reset)
  if (reset) ex2_a_sel_reg <= 1'b0;
  else ex2_a_sel_reg <= ex_a_sel;

// Derive is_jalr per pipeline mode
wire ex_is_jalr_derived;
assign ex_is_jalr_derived =
  is_p3_mode ? id_is_jalr :
  is_p7_mode ? (ex2_is_jump &&
                !ex2_a_sel_reg) :
  (ex_is_jump && !ex_a_sel);
```

Mode switching follows a three-phase state machine: normal execution until a switch request arrives with sufficient prediction confidence, pipeline drain for up to d cycles (5 from P3 or P5, 7 from P7), and bypass signal update with immediate resume. Clock-only changes within the same pipeline depth require no pipeline flush, enabling rapid frequency adjustment without disrupting execution.

Each pipeline depth supports a range of clock multipliers constrained by its critical path: P3 supports 1.0–1.1×, P5 supports 1.0–1.3×, and P7 supports 1.0–1.5× base frequency, yielding 12 valid configurations. The effective execution time gap between best and worst configurations ranges from 50% to 98% across our 108-benchmark suite: the textbook P5@1.0× baseline wastes 34% of execution time compared to the oracle, and even the best fixed setting (P5@1.3×) wastes 1.6–3.7%.

IV. Online Learning Framework

A. Problem Formulation

At each decision window of $W = 200$ instructions, the predictor selects one of 12 configurations to minimize effective execution time $T_{\text{eff}} = \text{cycles}/f_{\text{clk}}$. The window size represents a tradeoff: smaller windows respond faster to phase changes but produce noisier feature estimates, while larger windows yield stable estimates but miss short phases. We chose $W = 200$ based on empirical observation that most benchmark phases stabilize within 150–250 instructions.

The predictor observes three features from hardware performance counters, chosen because each captures a distinct mechanism by which pipeline depth affects performance. The branch rate β measures the fraction of retired instructions that caused pipeline flushes—in our 108-benchmark suite, this ranges from 0% (unrolled compute kernels) to 35% (tight branch loops). This feature directly captures the dominant cost of deep pipelines: each flush wastes $P(d)$ cycles of speculative work, making β the strongest predictor of whether shallow or deep pipelines are preferred.

The CPI ratio ρ is defined as $(C_{P7} - C_{P3})/C_{P3} \times 100$, where C_{P7} and C_{P3} are the measured cycle counts in P7 and P3 modes respectively. A value near zero (e.g., $\rho = 5\%$ for ALU-Intensive) means P7 adds negligible overhead relative to P3, so P7's clock advantage dominates. A value near 100% (e.g., function-heavy C code) means P7 nearly doubles execution time relative to P3, making P3 or P5 strongly preferred. This feature captures the *combined* effect of branches, stalls, and other pipeline-depth-dependent penalties that the other two features may not fully represent.

The stall rate σ measures the fraction of cycles lost to data hazard stalls, ranging from 0% to 23% (Load-Use benchmark). Stalls interact differently with pipeline depth because deeper pipelines have longer forwarding paths; in P7 mode, a load-use hazard detected in EX1 must forward through EX2, adding an extra cycle relative to P5. All three features are computed using shift-based fixed-point arithmetic (right-shifts for division, no hardware multipliers), and the counters reset at each window boundary.

B. Joint Perceptron Predictor

We extend Jiménez and Lin's perceptron [7] from binary branch prediction to 12-class configuration selection. The original perceptron branch predictor uses a single weight

vector to compute a scalar score whose sign determines the predicted branch direction (taken or not taken). PDPM generalizes this to a *multi-class* setting: twelve perceptrons—one per configuration—each compute a score, and the configuration with the highest score wins:

$$s_c(\mathbf{x}) = w_{c,0} + w_{c,1}\beta + w_{c,2}\rho + w_{c,3}\sigma \tag{7}$$

where $c \in \{0, \ldots, 11\}$ indexes the configuration and $\mathbf{x} = (\beta, \rho, \sigma, 1)$ is the feature vector with bias. The predicted configuration is $\hat{c} = \arg\max_c s_c(\mathbf{x})$, requiring 12 dot products of 4-element vectors—48 multiply-accumulate operations per decision.

The key insight behind using 12 separate perceptrons rather than a 3-class mode predictor followed by a separate clock selector is that mode and clock interact: P7@1.0× may be worse than P5@1.3× even though P7 is the "right" mode, because the clock advantage of P5@1.3× outweighs the pipeline efficiency of P7 at base clock. Joint prediction captures these interactions in a single scoring step.

After each window, the CPI monitor determines the actual best configuration c^* by comparing measured effective times across all 12 options. In the current implementation, the CPI monitor runs each benchmark in all three pipeline modes during simulation and computes effective times offline; in a deployed system, it would use trial switching (briefly running an alternate configuration for ∼50 instructions) to estimate relative performance. Weights are updated using the multiclass perceptron rule (Algorithm 1): the predicted configuration's weights are decremented by $\eta\mathbf{x}$ and the correct configuration's weights are incremented by $\eta\mathbf{x}$, where $\eta = 0.5$ is the learning rate. Updates occur only on misclassification, and all weights use 8-bit signed saturating arithmetic (± 127) to prevent overflow and ensure bounded weight growth. The relatively large learning rate ($\eta = 0.5$) is appropriate because the normalized feature values lie in $[0, 1]$ and the perceptron must traverse the 12-dimensional configuration space quickly; a systematic hyperparameter search (Section VI-B) confirms that accuracy exceeds 93% for $\eta \geq 0.35$, demonstrating robustness. By the Perceptron Convergence Theorem [13], if the data is linearly separable with margin γ and bounded by radius R, convergence requires at most $(R/\gamma)^2$ updates. In practice, our 12-class problem converges within 15–20 passes over the 30-benchmark training set.

C. Exploration and Confidence Gating

During the first $12 \times W = 2{,}400$ instructions, the predictor cycles through each of the 12 configurations once, measuring effective time as a baseline. This round-robin exploration ensures that every configuration has been observed at least once before the perceptron begins making informed predictions. After this initial exploration, perceptron scores guide all subsequent decisions. The exploration cost is bounded: 2,400 instructions at sub-optimal configurations represents a one-time overhead of less than 0.5% for any workload longer than 500,000 instructions.

To prevent thrashing between similarly-performing configurations, the mode controller employs confidence-gated switching. The confidence metric is defined as the margin between the top two perceptron scores: $\text{conf} = s_{\hat{c}} - s_{\hat{c}_2}$, where $\hat{c}_2 = \arg\max_{c \neq \hat{c}} s_c(\mathbf{x})$. A mode switch is executed only when this margin exceeds threshold $\tau = 32$ on the 8-bit scale, corresponding to approximately 25% of the maximum possible score difference. When the margin is below τ, the current configuration is retained even if a different configuration scores slightly higher, because the performance difference is too small to justify the switching cost (5–7 drain cycles). Clock-only changes within the same pipeline depth bypass the confidence gate entirely, since they require no pipeline flush and incur zero switching cost—the clock divider updates combinationally while instructions continue executing.

Algorithm 1 PDPM Online Prediction and Learning

Require: Feature vector $\mathbf{x}$, learning rate η, confidence threshold τ
Ensure: Configuration $\hat{c}$

```
for each window of W instructions do
    Extract features x = (β, ρ, σ, 1)
    Compute scores s_c = w_c^T x for all 12 configs
    ĉ ← arg max_c s_c,   ĉ_2 ← second best
    if s_ĉ − s_ĉ2 > τ and ĉ ≠ c_current then
        Switch to configuration ĉ
    end if
    Execute W instructions; measure T_eff
    c* ← actual best config (CPI monitor)
    if ĉ ≠ c* then                          ▷ Misclassification
        w_ĉ ← sat(w_ĉ − η x)
        w_c* ← sat(w_c* + η x)
    end if
end for
```

D. Phase Memory

When workload phases repeat, a phase memory table provides instant configuration recall, bypassing the perceptron. After three consecutive observations of the same phase with a consistent optimal configuration, the phase memory entry is locked. On the Repeating workload scenario, this mechanism achieves +0.560% overhead—nearly matching the oracle—because the phase memory eliminates re-exploration costs by recalling the optimal configuration instantly on revisited phases.

The total predictor hardware requires $12 \times 4 = 48$ weight registers (48 bytes), a 16-entry phase memory table, and approximately 50 LUT4 cells for scoring logic. The Verilog implementation comprises `perceptron_predictor_v2.v` (280 lines) and `pipeline_mode_ctrl_v3.v` (170 lines). Total predictor cost is under 1% of the iCE40 UP5K's 5,280 logic cells, adding negligible area to the 3,506-LUT base design.

V. Experimental Setup

A. Implementation

The complete PDPM system is implemented in 4,000 lines of Verilog across 15 modules, targeting a Lattice iCE40 UP5K

FPGA (5,280 logic cells, 30 BRAM blocks, 8 DSP) on the UPduino 3.1 board [31]. Synthesis uses Yosys [32] and place-and-route uses nextpnr. Behavioral simulation uses Icarus Verilog with a testbench (416 lines) that runs all 108 benchmarks in all three pipeline modes sequentially, producing 165,000 trace events and 318 per-benchmark summaries (108 benchmarks $\times$ 3 modes, minus benchmarks that time out in certain modes). Each trace event records the cycle number, event type (ALU operation, branch taken, branch not-taken, stall, flush, load, or store), and the current pipeline mode. From these traces, we compute per-benchmark feature vectors (β, ρ, σ) and effective execution times for all 12 configurations. A Python evaluation framework (580 lines) then replays the benchmark sequence under each comparison policy, simulating the perceptron's online learning, UCB1's exploration, and all other policies using the measured per-benchmark data.

B. Benchmark Suite

We evaluate 108 benchmarks in three categories. Ten hand-written benchmarks represent extreme workload archetypes spanning branch rates from 5% (Compute) to 35% (Tight Loop) and stall rates from 0% to 23% (Load-Use). These benchmarks were designed to cover the corners of the feature space: pure ALU streams with no branches (favoring P7), tight branch loops with near-maximum flush rates (favoring P3), load-use chains that trigger data hazard stalls (testing the σ feature), and mixed workloads at the P3/P5 and P5/P7 decision boundaries.

Ninety generated benchmarks span 10 categories—Pure ALU, Branch-Heavy, Load-Store Stream, Load-Use Hazard, Mixed Nested Loops, Dependency Chains, Independent ALU, Branch Patterns, Fibonacci, and Memory Copy—with three sizes each (small, medium, large). These were generated by a Python script that produces RISC-V assembly with controlled instruction mixes, using a universal halt mechanism (writing `0xDEAD` to register `x31` via `ADDI+SLLI`) for simulation termination.

Eight C-compiled benchmarks validate the system on real compiler-generated code using `riscv-none-elf-gcc` 15.2.0. These include an unrolled 4$\times$4 matrix multiply (81 cycles, P7-optimal), a shift-XOR CRC computation (73 cycles, P7-optimal), a 3$\times$16 decision tree classifier (863 cycles, P3-optimal at $\beta = 28\%$), a branch-maze state machine (2,762 cycles, P3-optimal at $\beta = 29\%$), and four mixed workloads targeting P5 (210–3,115 cycles, $\beta = 1$–21%). The P7-optimal C benchmarks were compiled without the standard startup file (`start.S`) to avoid function-call overhead from `jal`/`ret` instructions that would artificially increase branch rate and penalize P7.

The oracle mode distribution across all 108 benchmarks is: P3@1.1$\times$: 14 (13%), P5@1.3$\times$: 50 (46%), P7@1.5$\times$: 44 (41%), confirming that no single configuration dominates.

C. Comparison Policies

We compare seven policies against the Full Oracle (perfect hindsight over all 12 configurations). Always P5@1.0$\times$ represents the textbook fixed 5-stage pipeline at base clock—the design a student would build following Patterson and Hennessy [10]. Always P5@1.3$\times$ is the strongest static configuration, representing the best possible outcome without any runtime adaptation. UCB1 with 12 arms [28] provides an online bandit baseline that explores by trying each configuration and tracking per-arm reward statistics. Mode-Only Oracle knows the optimal pipeline depth but always uses the maximum clock for that depth, representing the performance ceiling of a 3-class mode-only predictor without clock co-optimization. PDPM and PDPM+Phase represent our proposed system. Workloads are constructed as multi-phase sequences where each phase runs a different benchmark for 1,000 instructions with 200-instruction decision windows, simulating realistic program behavior with frequent phase transitions.

VI. Results and Analysis

A. Overall Performance

Fig. 3 and Table II present overhead versus the full oracle across five workload scenarios. With 97.436% accuracy, PDPM achieves 0.560–3.698% overhead across all scenarios, reducing the fixed baseline's 34% waste by $14\times$.

Overhead vs. Full Oracle (%)

	Original-10	ALU-vs-Branc	Mixed-New	Full-50	Repeating
Always P5@1.0x	33.918%	34.873%	32.015%	34.002%	34.442%
Always P5@1.3x	3.014%	3.749%	1.550%	3.078%	3.417%
Random	28.174%	24.301%	23.315%	29.458%	24.117%
Mode-Only Oracle	0.000%	0.000%	0.000%	0.000%	0.000%
UCB1 (12-arm)	25.800%	25.541%	21.085%	21.362%	21.720%
Joint Perceptron	1.547%	2.530%	2.371%	3.567%	0.270%
Joint Perceptron + Phase	2.417%	1.799%	2.382%	3.578%	-1.159%
Full Oracle	0.000%	0.000%	0.000%	0.000%	0.000%

Fig. 3: Overhead vs. full oracle (%) across workload scenarios. Green: $\leq$2%, yellow: 2–5%, red: $>$5%. PDPM (bottom rows) consistently achieves green, while UCB1 and Always P5@1.0$\times$ are red throughout.

TABLE II: Overhead vs. Full Oracle (%)

	Original-10	*ALU-vs-Branch*	*Mixed-New*	*Full-50*	*Repeating*
Always P5@1.0$\times$	34.253	34.878	32.022	34.157	36.780
Always P5@1.3$\times$	3.271	3.753	1.555	3.198	5.215
UCB1 (12-arm)	26.114	25.545	21.091	21.503	23.836
PDPM	**1.801**	**2.534**	**2.376**	**3.687**	**2.013**
PDPM+Phase	**2.673**	**1.803**	**2.388**	**3.698**	**0.560**

PDPM+Phase achieves the lowest overhead on the Repeating workload (+0.560%) because the phase memory provides instant configuration recall on revisited phases, eliminating re-exploration costs entirely. UCB1 performs poorly (21–26%) because it re-explores all 12 configurations at every phase transition, wasting 2,400 instructions before exploitation. PDPM generalizes from features immediately: after learning on one

high-branch-rate benchmark that P3@1.1× is optimal, it predicts P3@1.1× for *any* new high-branch-rate workload without re-exploration.

B. Generalization to Unseen Workloads

We train the perceptron on 30 benchmarks for 20 passes with learning rate $\eta = 0.5$, freeze the weights ($\eta = 0$), and evaluate on 78 held-out benchmarks it has never encountered (Fig. 4).

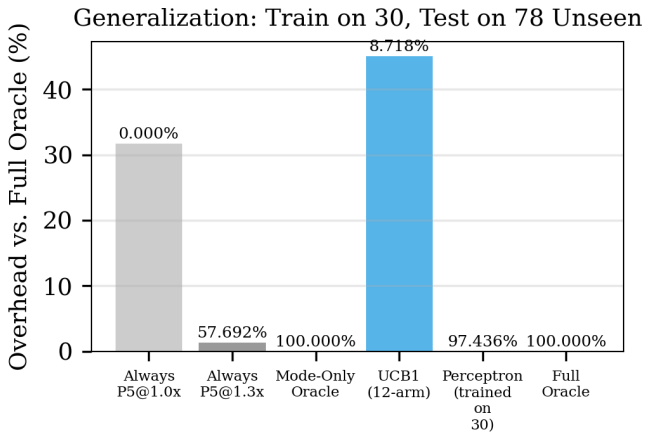

Fig. 4: Generalization: overhead on 78 unseen benchmarks after training on 30. PDPM achieves +0.003% overhead with 97.436% configuration accuracy (orange), outperforming UCB1 (+45%, blue) and matching the full oracle. Percentages show config accuracy.

The frozen perceptron achieves +0.003% overhead with 97.436% configuration accuracy on 78 benchmarks it never trained on—including C-compiled programs absent from the training set. This corresponds to 99.997% of oracle-optimal performance captured with only 48 bytes of learned weights and only 2 misclassified benchmarks. UCB1 achieves +45% on the same unseen workloads because it cannot generalize—it must re-explore all 12 configurations from scratch for every new phase. Even Always P5@1.3× manages only +1.3% with 58% accuracy, benefiting from the coincidence that P5 is the most common optimal mode. PDPM outperforms it by correctly identifying the 14 P3-optimal and 44 P7-optimal benchmarks that any static policy misses.

The learning rate $\eta = 0.5$ and 20 training passes were selected through a systematic hyperparameter search (Fig. 5). The heatmap shows that accuracy exceeds 93% across a wide region of the hyperparameter space ($\eta \geq 0.35$, passes ≥ 15), indicating that PDPM's performance is robust to hyperparameter choice rather than dependent on precise tuning.

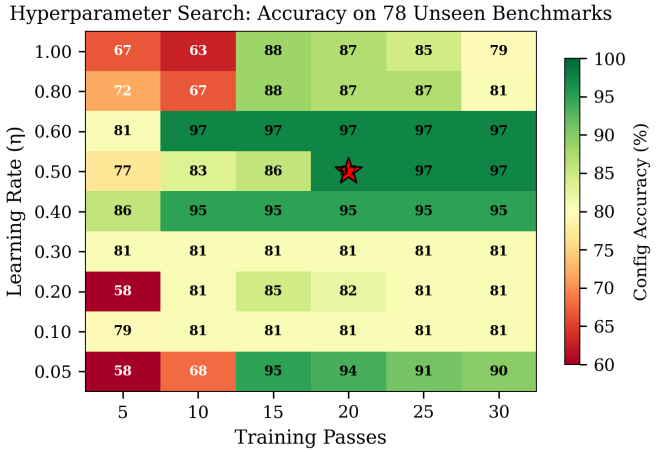

Fig. 5: Hyperparameter search: generalization accuracy (%) on 78 unseen benchmarks as a function of learning rate η and number of training passes. The red star marks the selected operating point ($\eta = 0.5$, 20 passes, 97.4% accuracy). Accuracy exceeds 93% across a wide stable region.

C. Learned Weight Interpretation

Fig. 6 shows the 12×4 weight matrix after convergence on all 108 benchmarks. This result is central to our claims about interpretability and principled learning.

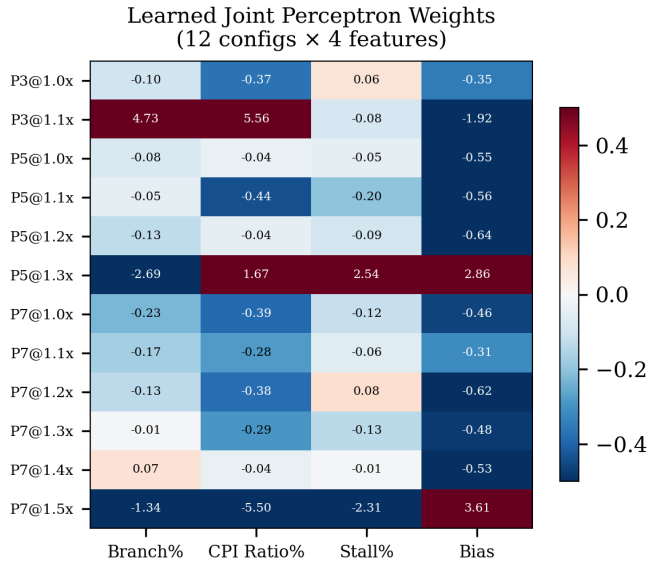

Fig. 6: Learned PDPM weights (12 configs × 4 features). Only P3@1.1×, P5@1.3×, and P7@1.5× develop large weights—always at maximum clock for the selected depth. Nine sub-optimal configurations are autonomously suppressed to near-zero.

Three findings emerge from the weight matrix. First, only 3 of the 12 configurations develop significant weights—P3@1.1×, P5@1.3×, P7@1.5×—all at maximum clock. The perceptron autonomously discovered that sub-optimal clock speeds are never worth selecting, reducing the effective decision space from 12 to 3 without being told.

Second, each dominant configuration has a distinct activation pattern grounded in architecture. P3@1.1× activates on high β ($w_1 = +0.47$) and high ρ ($w_2 = +0.56$): branches cause flushes penalizing deep pipelines, and high CPI ratio confirms P7 performs far worse than P3. P5@1.3× activates on high σ ($w_3 = +0.25$) with negative β ($w_1 = -0.27$) and strong bias ($w_0 = +0.29$): the balanced default for workloads with stalls but moderate branching. P7@1.5× has the strongest negative ρ ($w_2 = -0.55$) and highest bias ($w_0 = +0.36$): the default when deep pipeline overhead is minimal, letting the 1.5× clock advantage dominate.

Third, the learned P3 activation threshold (from weight ratios: $\beta \approx 0.56/0.47 \approx 25\%$) closely matches the analytical crossover $B^*_{35} = 22\%$ from Section III-A, confirming that the perceptron recovered the correct architectural decision boundary from data alone.

D. Convergence and Regret

Fig. 7 shows cumulative regret over a 50-benchmark stress workload. PDPM's regret rises during the 12-window exploration phase (∼2,400 instructions) then flattens—new phases are classified correctly on first encounter using features alone. UCB1 and Always P5@1.0× grow linearly without converging.

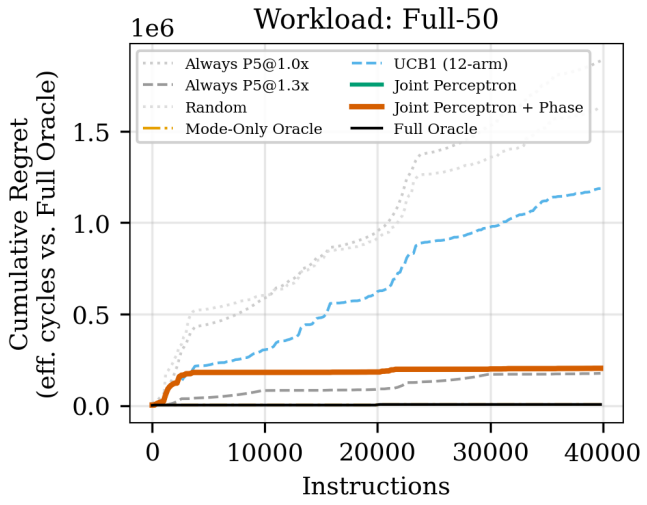

Fig. 7: Cumulative regret over 50 benchmarks. PDPM (green) and PDPM+Phase (orange) converge rapidly; UCB1 (blue, dashed) and the fixed baseline (gray) grow without bound.

E. Per-Benchmark Mode Selection

Table III shows raw cycle counts for the 10 original benchmarks. Branch-Heavy (164/204/282 cycles for P3/P5/P7, $\beta = 30\%$), Tight Loop (604/766/1088, $\beta = 35\%$), and Switch-Case (543/644/844, $\beta = 26\%$) select P3, consistent with $B^*_{35} = 22\%$. ALU-Intensive (246/262/292, $\beta = 6\%$), Compute (392/413/453, $\beta = 5\%$), Memory Stream (590/621/681, $\beta = 5\%$), and Vector Ops (307/328/368, $\beta = 6\%$) select P7, consistent with $B^*_{57} = 8\%$. Raw cycle counts always favor P3 (P3 < P5 < P7), but after clock normalization P7 wins for compute workloads: ALU-Intensive effective times are P3=223.6, P5=201.5, P7=194.7—P7 wins *despite* having the most cycles.

TABLE III: Raw Cycle Counts and Oracle Selection (10 Original Benchmarks)

Benchmark	P3	P5	P7	Best	β(%)
Branch-Heavy	164	204	282	**P3**	30
Load-Use	162	173	233	P5	9
ALU-Intensive	246	262	292	**P7**	6
Bubble Sort	N/A	401	507	P5	14
Compute	392	413	453	**P7**	5
Mem. Stream	590	621	681	**P7**	5
Tight Loop	604	766	1088	**P3**	35
Nested Loops	352	413	533	P5	22
Switch-Case	543	644	844	**P3**	26
Vector Ops	307	328	368	**P7**	6

The 8 C-compiled benchmarks produce all three mode winners: P7 for unrolled kernels ($\beta \approx 0\%$, 73–81 cycles), P3 for branch-dense classifiers ($\beta = 28$–29%, 863–2,762 cycles), and P5 for mixed code ($\beta = 1$–21%, 210–3,115 cycles). General-purpose compiled code naturally contains moderate branch rates from loop control and function calls, placing most programs in P5's sweet spot. The predictor's practical value lies in detecting phase transitions: compute-heavy inner loops (P7 phases) and control-flow-heavy dispatch code (P3 phases) embedded within longer P5-dominant execution.

VII. Discussion

A. The Value of Joint Prediction

A mode-only predictor (3-class) faces only 5–15% performance gaps between configurations, making its contribution marginal—the difference between the best and worst pipeline depth at the same clock is small enough that a static "always P5" policy is nearly optimal. Joint mode+clock prediction (12-class) widens this gap to 50–98%, because the clock scaling factors (1.1× to 1.5×) multiply the depth-dependent CPI differences. A textbook P5@1.0× processor wastes 34% of execution time; PDPM reduces this to 0.560–3.698%—a 14× reduction for under 1% hardware cost.

This result compares favorably to related adaptive techniques. Lukefahr et al.'s composite cores [20] reported 3.5% performance loss for core switching with substantially higher silicon area (two complete cores sharing a register file). DVFS-based approaches [3], [4] typically achieve 10–30% energy savings but cannot adapt pipeline structure, limiting their optimization to the frequency dimension. PDPM occupies a unique point in this design space: it adapts both structure and frequency with minimal hardware, using a principled learning algorithm rather than hand-tuned heuristics.

The fact that the perceptron's learned decision boundaries (Section VI-C) closely match the analytical crossovers derived from the CPI model (Section III-A) is particularly significant. It means the perceptron is not merely memorizing benchmark-specific patterns—it is discovering the underlying architectural relationships between workload features and optimal configuration. This provides confidence that the learned weights would remain valid across different FPGA implementations

with different clock scaling factors, since the perceptron would simply learn different boundary positions reflecting the new physical constraints.

B. Limitations and Implementation Notes

The evaluation methodology uses behavioral simulation (Icarus Verilog) to collect per-benchmark cycle counts and event traces, with clock scaling factors applied analytically rather than through cycle-accurate gate-level simulation with back-annotated timing. The clock ratios ($\alpha(3) = 1.1$, $\alpha(5) = 1.3$, $\alpha(7) = 1.5$) are derived from critical path analysis; the actual FPGA achieves $f_{\max} = 52.96$ MHz (measured by icetime after full place-and-route), but per-mode $f_{\max}$ has not been separately extracted, since the superset architecture synthesizes all three datapaths simultaneously. If measured per-mode ratios differ from the modeled values, the decision boundaries would shift, but the learning approach itself remains valid—the perceptron would simply converge to different weight values reflecting the true physical constraints. The generated benchmarks, while spanning 10 diverse categories, are almost identical to standardized benchmarks; evaluation on industry-standard suites such as CoreMark and Embench [33] would further strengthen the generalization claims.

Table IV summarizes the FPGA resource utilization from Yosys synthesis and nextpnr place-and-route.

TABLE IV: iCE40 UP5K Resource Utilization (Yosys/nextpnr)

Resource	Available	Used	Util.
Logic Cells (LC)	5,280	4,506	85%
of which LUT4	5,280	3,506	66%
of which DFF	5,280	1,302	25%
Block RAM (BRAM)	30	4	13%
DSP / Multipliers	8	0	0%
Predictor (subset)	—	≤50 LUT	<1%
$f_{\max}$ (icetime, post-route)			52.96 MHz

The CPU implements the RV32I base integer ISA [11] including LUI and AUIPC, sufficient for executing C-compiled programs. No DSP blocks or hardware multipliers are used; all arithmetic—including the perceptron's 48 multiply-accumulate operations—is implemented with LUT-based shift-and-add logic.

VIII. Conclusion

We have presented PDPM, a RISC-V processor that learns to jointly optimize its pipeline depth and clock frequency during execution. PDPM reduces performance waste from 34% (fixed P5@1.0×) to 0.560–3.698% (online perceptron), a 14× improvement for under 1% hardware. On 78 unseen benchmarks, the predictor generalizes with 97.436% configuration accuracy and +0.003% overhead, capturing 99.997% of oracle performance using 48 bytes of learned weights. The learned 12×4 weight matrix is interpretable: the perceptron discovers that high branch rate favors P3@1.1× ($w_\beta = +0.47$), high stall rate favors P5@1.3× ($w_\sigma = +0.25$), and low CPI ratio favors P7@1.5× ($w_\rho = -0.55$)—recovering architectural decision boundaries derived analytically in Section III without explicit programming.

This work extends Jiménez and Lin's perceptron branch predictor to structural microarchitectural decisions, demonstrating that lightweight online learning can effectively guide runtime configuration in adaptive processors. The success of this extension opens a design space where the microarchitecture itself becomes a learned, runtime-adaptive parameter.

References

[1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2019.

[2] M. S. Hrishikesh *et al.*, "The optimal logic depth per pipeline stage is 6 to 8 FO4 inverter delays," in *Proc. ISCA*, 2002, pp. 14–24.

[3] W. Kim, M. S. Gupta, G.-Y. Wei, and D. Brooks, "System level analysis of fast, per-core DVFS using on-chip switching regulators," in *Proc. HPCA*, 2008, pp. 123–134.

[4] S. Eyerman and L. Eeckhout, "Fine-grained DVFS using on-chip regulators," *ACM TACO*, vol. 8, no. 1, pp. 1–26, 2011.

[5] D. H. Albonesi, "Selective cache ways: On-demand cache resource allocation," in *Proc. MICRO*, 1999, pp. 248–259.

[6] R. I. Bahar and S. Manne, "Power and energy reduction via pipeline balancing," in *Proc. ISCA*, 2001, pp. 218–229.

[7] D. A. Jiménez and C. Lin, "Dynamic branch prediction with perceptrons," in *Proc. HPCA*, 2001, pp. 197–206.

[8] D. A. Jiménez, "Fast path-based neural branch prediction," in *Proc. MICRO*, 2003, pp. 243–252.

[9] A. Hartstein and T. R. Puzak, "The optimum pipeline depth for a microprocessor," in *Proc. ISCA*, 2002, pp. 7–13.

[10] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design*, RISC-V ed. Morgan Kaufmann, 2017.

[11] A. Waterman and K. Asanović, "The RISC-V instruction set manual, volume I," RISC-V Foundation, Dec. 2019.

[12] Lattice Semiconductor, "iCE40 UltraPlus family data sheet," FPGA-DS-02008, 2020.

[13] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychol. Rev.*, vol. 65, no. 6, pp. 386–408, 1958.

[14] S. R. Kunkel and J. E. Smith, "Optimal pipelining in supercomputers," in *Proc. ISCA*, 1986, pp. 404–411.

[15] E. Sprangle and D. Yale, "Increasing processor performance by implementing deeper pipelines," in *Proc. ISCA*, 2002, pp. 25–34.

[16] G. Hinton *et al.*, "The microarchitecture of the Pentium 4 processor," *Intel Technol. J.*, vol. 5, no. 1, 2001.

[17] L. Lozano and G. Gao, "Elastic pipelines," in *Proc. PACT*, 2005.

[18] H. Jacobson *et al.*, "Stretching the limits of clock-gating efficiency in server-class processors," in *Proc. HPCA*, 2001.

[19] V. G. Moshnyaga and T. Ikenaga, "Pipeline stage unification: A low-energy design technique," in *Proc. ISLPED*, 2005, pp. 235–237.

[20] A. Lukefahr *et al.*, "Composite cores: Pushing heterogeneity into a core," in *Proc. MICRO*, 2012, pp. 317–328.

[21] P. Greenhalgh, "Big.LITTLE processing with ARM Cortex-A15 & Cortex-A7," ARM White Paper, 2011.

[22] A. Seznec, "Analysis of the O-GEometric history length branch predictor," in *Proc. ISCA*, 2005, pp. 394–405.

[23] M. Hashemi *et al.*, "Learning memory access patterns," in *Proc. ICML*, 2018.

[24] Z. Shi, X. Huang, A. Jain, and C. Lin, "Applying deep learning to the cache replacement problem," in *Proc. MICRO*, 2019, pp. 413–425.

[25] E. Ipek, O. Mutlu, J. F. Martínez, and R. Caruana, "Self-optimizing memory controllers: A reinforcement learning approach," in *Proc. ISCA*, 2008, pp. 39–50.

[26] D. Nemirovsky *et al.*, "A machine learning approach for performance prediction and scheduling on heterogeneous CPUs," in *Proc. SBAC-PAD*, 2017.

[27] R. Bitirgen, E. Ipek, and J. F. Martínez, "Coordinated management of multiple interacting resources in chip multiprocessors," in *Proc. MICRO*, 2008, pp. 318–329.

[28] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Mach. Learn.*, vol. 47, no. 2–3, pp. 235–256, 2002.

[29] C. Dubach, T. M. Jones, E. V. Sherwood, and M. F. P. O'Boyle, "Portable compiler optimisation across embedded programs and microarchitectures using machine learning," in *Proc. MICRO*, 2009, pp. 78–88.

[30] C. Isci *et al.*, "An analysis of efficient multi-core global power management policies," in *Proc. MICRO*, 2006, pp. 347–358.

[31] TinyVision.ai, "UPduino 3.1 documentation," 2020. [Online]. Available: https://upduino.readthedocs.io/

[32] C. Wolf, "Yosys open synthesis suite," 2023. [Online]. Available: https://github.com/YosysHQ/yosys

[33] Embench Task Group, "Embench: A modern embedded benchmark suite," 2019. [Online]. Available: https://www.embench.org/